PART VI — THE SELF-LEARNING LAYER

Chapter 36C

Evidence-Based Retrieval Tuning from Production Logs

"A threshold chosen by reasoning alone is a hypothesis. Only production logs turn it into a finding."

Chapter 36C — Evidence-Based Retrieval Tuning from Production Logs

This chapter is the most log-grounded in the book, because the case study behind it is unusually well documented in this project's own research ledger: three named configurations, two representative queries, and exact LLM-call-count deltas recorded for each — a genuine A/B comparison with a result that overturned the config a shallower metric would have chosen.

36C.1 Why hand-picked thresholds fail

Chapter 33.8's `DOCUMENTS_MIN_SIMILARITY = 0.53` and `LEARNED_QA_MIN_SIMILARITY = 0.57` did not start at those values — this project's initial deployment used a single, conservative `MIN_SIMILARITY = 0.5` floor for both collections, chosen the way most first thresholds are chosen: a reasonable-sounding round number, picked before any real usage data existed to calibrate it against. A hand-picked threshold is not wrong to start with — Research topic 39 explicitly frames the conservative 0.5 starting point as the correct initial choice specifically *because* no evidence base yet existed — but treating it as a permanent value rather than a starting hypothesis is the actual failure.

The knobs that most need this recalibration share a common property: their correct value depends on the *actual* score distribution a specific corpus and a specific embedding model produce, which is empirical information no amount of reasoning about cosine similarity in the abstract can substitute for. `RETRIEVAL_TOP_K`, `RETRIEVAL_TOP_L`, and both per-collection similarity floors are all in this category — reasonable to estimate initially, but genuinely wrong to leave uncalibrated once real traffic exists to calibrate them against.

This is worth distinguishing sharply from constants where hand-picking genuinely is the right permanent answer. `MAX_ITERATIONS`, Chapter 23's iteration cap, is chosen against a fixed, external constraint — Groq's rate limit and this project's own token-budget math — that does not shift based on what the corpus happens to contain. A retrieval threshold's correct value, by contrast, is a direct function of the embedding model's actual output distribution for actual queries and actual chunks, which no engineer can derive from first principles with any real precision. The dividing line is whether the constant is anchored to an external, stable fact or to an empirical distribution only production traffic can reveal.

36C.2 The A/B log-comparison methodology

> **Definition — A/B log comparison**
> Running an identical query set through two or more candidate configurations and comparing the resulting debug logs on multiple axes — call count, verdict quality, and answer correctness — rather than trusting any single metric in isolation to pick a winner.

This project's own `extract_logs.py` — a 443-line standalone script — parses `app_workflow/run_logs/` debug logs and extracts exactly the fields a config comparison needs: collection query parameters, per-chunk cosine scores, validation verdicts, and LLM call counts. The methodology built around it, per Research topic 40, is precise: select one simple in-domain query and one complex multi-domain query, execute each under every candidate configuration, and compare the resulting logs on three axes simultaneously rather than one.

Using exactly two query types — one simple, one deliberately complex — is a deliberate minimal design, not an oversight. A single query type risks tuning toward whatever that one type happens to reward; a simple and a complex query together are enough to reveal the specific failure mode Section 36C.4 covers, where a configuration that looks strictly better on an easy case turns out to starve a harder one.

It is worth being precise about what makes this a genuine A/B comparison rather than merely running the pipeline twice. Every other variable — the corpus, the embedding model, the LLM, the prompt templates — stays fixed across all three configurations tested; only the specific config values named in each candidate change. Without that discipline, an observed difference in call count or correctness could be caused by anything that happened to differ between the two runs, not necessarily the configuration values under test. Fixing every variable except the one being evaluated is what turns a pair of log files into actual evidence rather than two anecdotes.

36C.3 Choosing RETRIEVAL_TOP_K and RETRIEVAL_TOP_L from observed distributions

Research topic 39's log analysis found the signal directly: at the pre-tuning default of 5 chunks per collection, validation (`validate_document_retrieval` / `validate_learned_qa_retrieval`, Chapter 11's relevance judge applied per track) consistently dropped 2-3 chunks as irrelevant, leaving only 2-3 genuinely useful chunks per collection reaching compression. The fifth-ranked chunk was below threshold or irrelevant in the majority of observed runs — meaning the pipeline was paying for a retrieval, an embedding comparison, and a validator judgment on a chunk that got discarded often.

Reducing to `RETRIEVAL_TOP_K = 4` and `RETRIEVAL_TOP_L = 4` was the direct, evidence-based response: fewer chunks retrieved per call, minimal recall loss (since the discarded fifth chunk was already usually being rejected by validation anyway), and a smaller validator workload per retrieval — Chapter 23.4's token math made concrete by production log evidence rather than by reasoning about chunk counts in the abstract.

It is worth naming why this particular signal — the fraction of retrieved chunks a downstream validator subsequently rejects — is such a clean way to right-size `top_k` specifically. `validate_retrieval()` (Chapter 11) already runs on every retrieval call regardless of whether anyone is tuning `top_k` at all, which means the evidence this section relies on was not collected by any special instrumentation built for this analysis — it was sitting in the existing debug logs the whole time, in the ordinary `[VALIDATE-RETRIEVAL]` verdict lines every run already produces. Evidence-based tuning did not require building new measurement infrastructure here; it required someone to go back and read what the infrastructure already in place had been recording.

36C.4 Choosing the per-collection similarity floors

The full three-configuration comparison — `K=5/0.50/0.50` (baseline), `K=4/0.55/0.60` (aggressive), `K=4/0.53/0.57` (intermediate) — is where this chapter's case study earns its place as the clearest demonstration of why call count alone is an insufficient metric. On the simple, single-domain query, LLM call counts dropped from 148 (baseline) to 42 (aggressive) to 30 (intermediate) — a result that, read in isolation, would make the aggressive configuration look like the obvious winner.

The complex, three-variant "ASD" query told a different story entirely: baseline used 205 calls with 3 INSUFFICIENT verdicts and 2 retrieval retries; the aggressive config dropped to 37 calls with zero INSUFFICIENT verdicts, but 2 of 3 query variants retrieved zero chunks at all — the higher similarity floor was starving retrieval on a genuinely harder, multi-domain query, and the model filled the resulting gap by hallucinating a domain meaning for "ASD" rather than admitting insufficient evidence. The intermediate config used 30 calls, zero INSUFFICIENT verdicts, all three variants retrieved real content, and the answer was correct.

```
Simple query  — LLM calls: baseline 148 -> aggressive 42 -> intermediate 30
Complex query — LLM calls: baseline 205 -> aggressive 37  -> intermediate
30
Complex query — variants starved (0 chunks): baseline 0 -> aggressive 2/3 -
> intermediate 0
Complex query — INSUFFICIENT verdicts:       baseline 3 -> aggressive 0   -
> intermediate 0
```

Figure 36C.1 lays these three configurations side by side specifically so the trap in reading only the top row is visible at a glance — call count alone ranks the aggressive configuration ahead of the intermediate one on both queries, and only the starvation and verdict rows beneath it reveal why that ranking would have been the wrong one to ship.

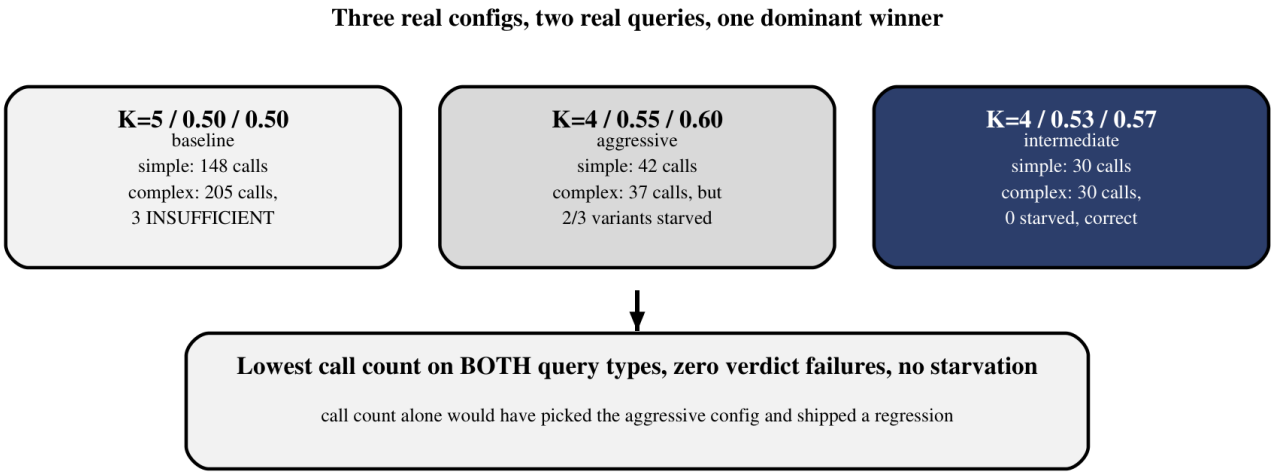

Figure 36C.1 — The aggressive config wins on call count alone; only checking correctness on the harder query reveals it was starving retrieval, not genuinely improving efficiency.

The precise language Research topic 40 uses for this finding is worth repeating exactly: "LLM call count is a necessary but insufficient metric — answer correctness on representative multi-domain queries must also be verified, because a threshold that is too high will silently starve retrieval and cause hallucination rather than routing to `no_context_answer`." A cleaner failure would have been the system honestly admitting it had no answer (Chapter 8's `NO_CONTEXT_ANSWER` path); the actual failure was worse — confident, fluent, wrong.

This failure mode connects directly back to Chapter 24's long-context reliability zones and Chapter 25's account of why a small model under-resourced on evidence tends to fill the gap with something plausible-sounding rather than an honest admission of insufficiency. A similarity floor set too high does not make the model more careful — it starves the model of the very evidence that would have let it be careful, and an 8B model denied real evidence for a genuinely answerable-seeming question does not reliably default to silence.

36C.5 Detecting silent regressions

The aggressive configuration's failure is the template for a broader risk this chapter's methodology exists to catch: a change that looks like a strict improvement on every metric someone happened to check can still be a regression on a dimension nobody checked. A "cleanup" refactor — reordering a validation step, adjusting a default without re-running the A/B comparison, simplifying a scoring formula — can silently shift retrieval behavior in exactly this way, passing every existing automated test (none of which typically assert on *which* chunks got retrieved, only that retrieval returned *something*) while quietly degrading real answer quality on the harder query types those tests never happened to include.

This is precisely why Chapter 36B.6's before/after chunk-set comparison and this chapter's A/B methodology are the same discipline applied to two different kinds of change — a config value and a structural refactor. Neither kind of change is safe to ship on the strength of "the tests still pass" alone when the tests in question were never designed to catch a shift in retrieval quality specifically.

A useful operational habit follows directly from this: treat any change touching `retriever.py`, the collection query parameters, or any of the four threshold constants this chapter covers as requiring the Section 36C.2 methodology before merge, not as a nice-to-have follow-up. A pull request that changes `DOCUMENTS_MIN_SIMILARITY` by a few hundredths, reviewed only by reading the diff, looks trivial — one number changed. Whether that trivial-looking diff starves retrieval on the next multi-domain query the way the aggressive configuration did is not something the diff itself can answer; only running the fixed query pair through it and checking the resulting logs can.

> **Common pitfall — Trusting green tests to catch a retrieval regression**
>
> A test suite asserting that `retrieve_documents` returns a non-empty list will pass whether that list contains the five most relevant chunks in the corpus or five barely-relevant ones. Retrieval quality regressions need the A/B methodology this chapter builds, not unit-test coverage, because the failure mode is a change in *which* correct-shaped result comes back, not a crash or an empty response.

36C.6 Building a repeatable tuning loop

The methodology this chapter has walked through generalizes into a loop, not a one-time project: maintain the fixed simple-and-complex query pair, run it under any new candidate configuration, capture a timestamped debug log the way this project's `app_workflow/run_logs/` convention already does, and diff the result against the current production configuration using `extract_logs.py` before ever changing a default in `config.py`.

Figure 36C.2 draws this loop as a cycle rather than a line for a specific reason: the fourth step's output — an evidence-based decision — feeds directly back into the first step the next time a config change is even being considered, rather than terminating the process once `K=4/0.53/0.57` shipped. The loop is the reusable asset; today's specific numbers are simply its most recent output.

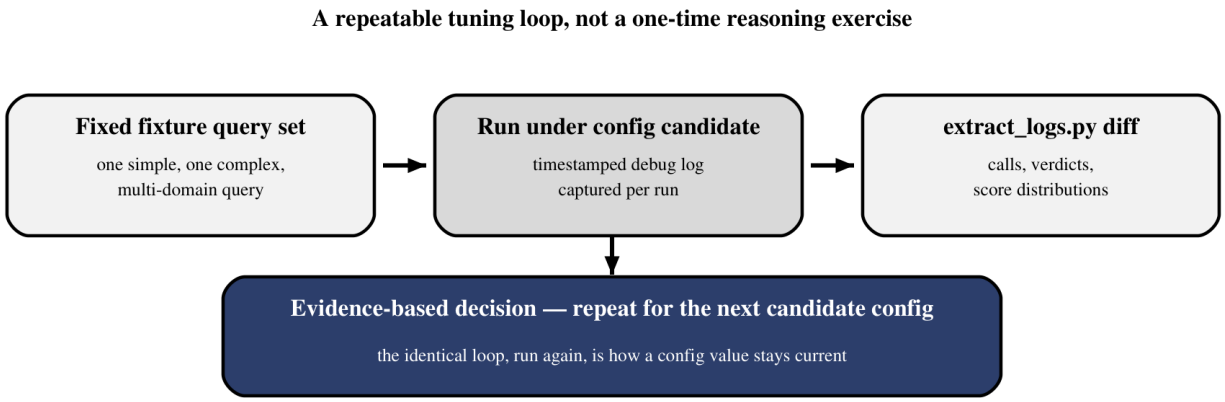

Figure 36C.2 — The same four-step loop applies to every future config candidate; repeatability, not a single successful run, is what makes evidence-based tuning durable.

Research topic 40 states this repeatability explicitly as the intended legacy of the work: "the methodology is repeatable: run the same query pairs against any future config candidate, compare debug logs with `extract_logs.py`." A configuration value tuned once and never revisited drifts back toward being a hand-picked guess the moment the corpus, the embedding model, or the query patterns it was tuned against change — the loop, not the specific numbers `K=4/0.53/0.57` happen to represent today, is this chapter's actual, durable contribution.

Part VI closes here, on exactly the discipline it opened with in Chapter 29: nothing in this self-learning layer — not the feedback ledger, not the failure blocklist, not distillation, not retrieval tuning — is trustworthy by assertion alone. Every mechanism this part of the book built earns its place in production the same way, through a fixed reference point, a controlled comparison, and a willingness to let real logs overturn a reasonable-sounding guess. Part VII turns to what it takes to run all of it, together, in production.

Looking back across all ten chapters of this part, the throughline is the same one this final chapter states most explicitly: a system that can learn from its own history — failed queries, thumbdowns, verified successes, retrieval logs — is only as trustworthy as its own willingness to be measured against that history rather than merely accumulating it. Chapter 29's ledger, Chapter 32's distillation gate, and this chapter's threshold evidence are three instances of the identical discipline, applied to three different kinds of accumulated experience, and none of them would mean anything without the fixed baselines and repeatable comparisons that turn raw accumulation into an actual, falsifiable claim of improvement.